

2. O Processo de Desenvolvimento de Software

Objetivos:

Este capítulo aborda as diversas maneiras de como um software deve ser desenvolvido. Veremos o conceito de *ciclo de vida*, identificando suas principais fases e as atividades do ciclo de vida do software. Finalizaremos com o estudo de diversas propostas e modelos para o processo de desenvolvimento de software.

2.1 Processo de desenvolvimento

Vimos na [introdução](#) que uma engenharia de software requer que as atividades para desenvolver o software sejam feitas de forma planejada, gerenciada, com pessoal capacitado, custos e prazos estimados e utilizando teorias, métodos, técnicas e ferramentas adequadas.

Elaborar um **processo de desenvolvimento de software** significa determinar de forma precisa e detalhada quem faz o que, quando e como. Um processo pode ser visto como uma instância de um método com suas técnicas e ferramentas associadas, elaborado durante a etapa de planejamento, no qual as atividades que o compõem foram alocadas aos membros da equipe de desenvolvimento, com prazos definidos e métricas para se avaliar como elas estão sendo realizadas (veja [conceitos básicos](#)).

Enquanto um método é algo teórico, o processo deve determinar ações práticas a serem realizadas pela equipe como prazos definidos. O processo é o resultado do planejamento e precisa ser gerenciado no decorrer de sua execução.

Não é objetivo deste capítulo a elaboração de processos de desenvolvimento. Apenas podemos fazê-lo após estudarmos técnicas de planejamento e gerenciamento. Neste capítulo vamos nos limitar a estudar alguns modelos de processo que nos indique como as diversas etapas e atividades do desenvolvimento podem ser estruturadas.

2.2 O ciclo de vida do software

O ciclo de vida de um artefato diz respeito às diversas fases pelas quais ele passa desde o seu surgimento até a o momento no qual ele não será mais útil. No sistema computacional todos os componentes possuem um ciclo de vida próprio. Embora eles sejam relacionados entre si, eles possuem ciclos de vida independentes. O hardware pode continuar existindo mesmo que o software seja destruído. O software, em sua forma estática, pode também continuar existindo mesmo que um computador específico torne-se inoperante. Quando o software é construído para um hardware específico é que o ciclo de vida de ambos pode ser mesmo. Neste curso, vamos nos limitar a estudar e discutir o ciclo de vida do software.

No ciclo de vida do software identificamos três fases:

- **Definição**
- **Desenvolvimento**
- **Operação**

Na fase de **definição** os requisitos do software são determinados, a sua viabilidade é estudada e o planejamento das atividades é elaborado.

Na fase de **desenvolvimento** são realizadas as atividades destinadas a produção do software. Ela envolve atividades de concepção, especificação, design da interface, prototipação (do inglês *prototyping*, traduzido também por prototipagem), design da arquitetura, codificação e verificação, dentre outras.

Na fase de **operação** o sistema deverá efetivamente ser utilizado pelos seus usuários produzindo os resultados desejados. Nesta fase devem ocorrer as atividades de manutenção, seja para que se façam correções, ou seja para a sua evolução, isto é, para que o software satisfaça novos requisitos.

Nas próximas seções, vamos descrever algumas das principais atividades do ciclo de vida do software. O nosso objetivo é essencialmente didático. Vamos apresentá-las para que possamos compreender os diferentes modelos do processo que serão apresentados na próxima seção. Algumas destas atividades podem ou não aparecer em um determinado modelo, bem como pode existir alguma atividade específica de um certo modelo que não será mencionada aqui.

Optamos por apresentar as atividades do desenvolvimento classificando-as por fase do ciclo de vida. Entretanto, dependendo do modelo de processo algumas atividades também podem estender-se por mais de uma fase do ciclo de vida.

2.2.1 As atividades da fase de definição

Como normalmente o software está inserido num contexto mais amplo - o sistema - a fase de definição do software está inserida na definição do sistema. Definir o sistema é definir todos os seus componentes ([ver capítulo 1](#)).

Na fase de definição são tomadas as decisões de construir ou não o software. Nela são definidos os **requisitos do software**, determinando-se o que o cliente quer, o que a organização necessita, quais os problemas nas atividades dos usuários, etc. O cliente pode, por exemplo, necessitar de um software que faça o controle de vendas e compras da sua empresa. Um cliente de uma empresa de publicidade pode necessitar de um software de editoração eletrônica para a *Web*.

Também devem ser definidas algumas restrições ao software. Um exemplo de restrição técnica é: "o software deve ser executado no ambiente *Unix/XWindow*, uma vez que esta é a plataforma instalada na empresa". Outro caso seria: "o software deve ser executado num sistema remotamente distribuído uma vez que a empresa possui diversos pontos de venda". Existem restrições econômicas: "o orçamento de desenvolvimento não pode ultrapassar R\$ 10.000,00".

A definição dos requisitos é denominada de **análise e especificação de requisitos** indicando que existe uma atividade de observação e uma descrição rigorosa dos problemas e da proposta de soluções. Alguns autores argumentam que devido à sua complexidade esta atividade deve ser considerada uma **engenharia de requisitos**. O objetivo deles também é ressaltar que os requisitos devem ser gerenciados durante todo o ciclo de vida do software.

O termo [especificação](#) está associado a diversas atividades do ciclo de vida: especificação de requisitos, especificação do software, especificação da arquitetura, especificação de dados e algoritmos, etc. e se refere a descrição, através de alguma notação, de algo que foi concebido ou idealizado. A especificação de requisitos tem por objetivo descrever aquilo que clientes ou futuros usuários necessitam e que será resolvido pela construção de um software. Como esta especificação precisa ser validada por clientes e usuários, normalmente ela é feita através de uma notação informal, como a linguagem natural, ou usando uma linguagem gráfica semi-formal como UML, DFD, DER, etc. Uma técnica bastante utilizada para especificação informal de requisitos são os **cenários** ([ver capítulo 3](#)).

O resultado desta atividade é a descrição dos **requisitos funcionais** - que dizem respeito àquilo que se quer que o software faça - e os **não-funcionais** - que dizem respeito a requisitos de ordem técnica, econômica, da organização, etc.

Nesta fase também deve haver um **estudo de viabilidade** do software. Este estudo visa verificar se o software é viável **técnica e economicamente**, e se os benefícios trazidos serão compensadores. O estudo de viabilidade requer que tenham sido definidos alguns requisitos para que se possa ter idéia do que será o

sistema. Conforme veremos mais adiante, alguns modelos de processo podem determinar que o estudo de viabilidade seja feito apenas após a análise completa dos requisitos. Em outros, ele pode ser realizado simultaneamente ou num processo iterativo.

Associada a esta atividade devem ser realizadas **estimativas de custos e prazos** e a **análise de riscos**. A primeira visa determinar gastos e prazos aproximados a partir de dados de experiências anteriores. A análise de riscos visa verificar se existem possibilidades de que algo possa sair errado, como por exemplo, o orçamento estourar ou se haverá dificuldades técnicas.

O resultado do estudo de viabilidade é a decisão de que o software deve ou não ser construído com base nos requisitos, nas restrições técnicas, nas estimativas de custos e análise de riscos, dentre outros fatores, em relação aos benefícios que o sistema deverá proporcionar.

Também na fase de definição, caso o software seja viável, deve ser feito o **planejamento** de como o desenvolvimento será conduzido, isto é, deve-se elaborar um processo de desenvolvimento. O planejamento não necessariamente precisa ser feito completamente nesta fase. Veremos modelos do processo no qual o planejamento é revisto diversas vezes ao longo do desenvolvimento. O resultado do planejamento é uma descrição precisa do [processo de desenvolvimento de software](#).

Existem diversas propostas sobre quem deve realizar as atividades na fase de definição. Na maior parte dos casos a definição dos requisitos iniciais é conduzida por **analistas de sistemas** como parte da definição do sistema. Isto significa que a fase de definição do software pode ser considerada como parte da análise de sistema que visa definir o sistema computacional do qual o software é parte. Para contribuir como definições mais específicas sobre o software, como estimativas de custos e prazos e o planejamento do desenvolvimento de software, **engenheiros de software** e analistas de sistemas devem trabalhar conjuntamente. Em diversas empresas analistas de sistemas realizam mais do que a definição dos requisitos do sistemas. Eles também definem e projetam o software.

2.2.2 Atividades da fase de desenvolvimento

Um **modelo do processo de desenvolvimento** ou **método** compreende algumas atividades. Cada proposta de modelo apresentado na literatura acadêmica ou elaborado por empresas de desenvolvimento devem determinar quais as atividades compõem o método. Embora estas atividades variem de uma proposta para outra, algumas são comuns a vários métodos. Além disso, em alguns modelos de processo, algumas atividades da fase de definição também podem estar inseridas nesta fase para que possam ser revisadas. Existem modelos que propõem ciclos cujas atividades da fase de desenvolvimento são alternadas com atividades da definição.

Design

Design é a atividade de concepção e [especificação](#) de um produto. A concepção é a atividade mental de criação do produto que satisfaça aos requisitos. Aquilo que foi concebido deve ser concretizado na forma de uma [especificação](#). Muitos autores utilizam o termo especificação no sentido de design. Design pode ser traduzido como projeto ou como desenho. Consideramos que ambas não são traduções ideais e vamos optar por usar a palavra em inglês.

Diversos autores argumentam que se o software deve ser visto como um produto a atividade de design de software é imprescindível [Winograd 96]. Assim, **design de software** é atividade do desenvolvimento na qual o software deve ser concebido e especificado do ponto de vista do usuário e não do desenvolvedor. O foco está na visão externa do software que é a aquela que será percebida pelos usuários. Fazendo uma analogia com a engenharia civil, o design de software seria uma atividade equivalente à arquitetura de uma edificação.

O design deve determinar o que o software deve fazer - a sua **funcionalidade** - e como o usuário irá interagir como ele - a sua **interatividade** ou modelo de interação. Estes dois elementos compõem o **modelo conceitual da aplicação**. Desta forma o design de software envolve a concepção e especificação da funcionalidade e do modelo de interação.

O design toma como base a especificação de requisitos elaborada na fase definição. É importante ressaltar que, dependendo do processo de desenvolvimento, a especificação dos requisitos pode estender-se pela fase de desenvolvimento, como veremos mais adiante. Enquanto que a especificação dos requisitos visa descrever o que clientes, usuários e organização necessitam, o design de software visa especificar o que ele oferecerá para satisfazer estas necessidades.

Existem decisões importantes que são tomadas durante o design de software que fogem ao escopo de uma especificação de requisitos. Por exemplo, para o requisito funcional de "o software deve realizar o cálculo do total de vendas e do lucro obtido", o designer do software deve decidir os dois cálculos serão feitos simultaneamente por uma única função ou se por duas funções independentes. Ele deve decidir se o usuário fornece os dados todos inicialmente e apenas ao final os cálculos são feitos ou se os dados são fornecidos para cada cálculo que se deseja fazer.

Uma vez que o designer tenha especificado a funcionalidade e o modelo de interação ele pode apresentar aos clientes e usuários para as suas opiniões ou rever os requisitos. Quando os requisitos são refinados a partir da especificação do software as atividades de especificação de requisitos e de software se confundem. Nestas condições nas quais o design é realizado com a participação dos clientes e usuários ele é chamado de **design participativo**.

A engenharia de software tradicionalmente enfatiza apenas na especificação da funcionalidade. A proposta de design centrado no usuário visa chamar a atenção para o modelo de interação e para a interface de usuário como elementos importantes na qualidade de um software.

O **design da interface de usuário** visa a concepção e especificação da parte do software que possibilita que o usuário interaja com o sistema de acordo com o modelo de interação especificado. Enquanto o modelo de interação determina os protocolos de comunicação entre o usuário e o sistema, a interface deve apresentar os menus, janelas, ícones, botões que permitem as ações do usuário de acordo com o modelo. O design da interface é a concretização de um modelo de interação especificado durante o design do software.

A maioria dos autores de engenharia de software utilizam o termo de design do software para se referir à especificação da arquitetura de software, isto é, da configuração de componentes do software - funções, classes, objetos e suas interconexões. Neste caso, estamos realizando design do ponto de vista do desenvolvedor e chamaremos de **design da arquitetura do software**. O design da arquitetura visa determinar de maneira abstrata como a funcionalidade será implementada.

O design da arquitetura deve resultar na especificação da abstrata da macro-estrutura dos componentes do software e de como eles interagem entre si, sem preocupação com detalhes a respeito dos algoritmos que descreverão o funcionamento de cada componente. No design arquitetural é importante que se especifique o que cada componente deve fazer - **especificação funcional de componentes**.

O **design de algoritmos e dados**, também chamado de design detalhado, tem por objetivo a concepção e especificação das estruturas de dados e dos algoritmos que realizam aquilo que foi especificado para cada componente do software. A especificação funcional dos componentes deve se feita a nível do que chamamos de *problemas algorítmicos*, isto é que podem ser resolvidos pela construção de um algoritmo. As soluções algorítmicas são normalmente encontradas na literatura especializada em design de algoritmos e de estruturas de dados.

Em alguns casos, pode-se encontrar componentes de software já desenvolvidos em código fonte ou executável - em bibliotecas, por exemplo - e incorporá-los durante a programação.

O resultado de todas as atividades de design são especificações da funcionalidade, do modelo de interação, da interface de usuário, da arquitetura de software e dos algoritmos e estruturas de dados.

Prototipação

Uma outra forma de concretizar a concepção de um software é a através de um protótipo. Um protótipo é um produto construído utilizando materiais mais baratos e com dimensões reduzidas. A **prototipação** (ou

prototipagem) é atividade de construir protótipos. Existem modelos de processo de desenvolvimento que são baseados em prototipação como veremos a seguir.

Um protótipo do software é construído utilizando ferramentas que permitem que apenas partes do software sejam construídas com o objetivo de verificar suas qualidades antes que o produto final venha a ser construídos.

A prototipação, uma técnica usada freqüentemente nas engenharias, tem sido adotada na engenharia de software para aperfeiçoar as previsões e diminuir os riscos envolvidos no desenvolvimento de um novo projeto. Os termos protótipos e prototipação têm sido usados na literatura com diversos significados e não se definiu uma classificação ou critérios para tal, de maneira convincente.

Programação

A programação consiste na atividade de construção de um programa que implementa uma determinada solução para um problema algorítmico. Esta solução pode ser descrita na forma de especificação de algoritmos e estruturas de dados e deve então ser codificada uma linguagem de programação. Especialistas em programação podem escrever a solução algorítmica diretamente na linguagem de programação.

Esta atividade é também chamada de implementação, construção ou codificação do software. A codificação enfatiza a que o software deve ser construído utilizando uma linguagem de programação. Além da codificação propriamente, a programação deve envolver ainda a sua tradução num código de máquina executável.

A programação deve resultar num programa que implemente tudo o que foi especificado durante o design. No caso de software formado por vários componentes, estes devem ser implementados a partir da especificação de cada componente e **integrados** (interconectados) conforme a arquitetura interna do software.

Avaliação ou Verificação

A avaliação ou verificação visa assegurar algumas das principais qualidades do software. Dentre as atividades de avaliação vamos destacar a correção, validação e usabilidade do software.

O software é considerado correto quando o programa implementado satisfaz à sua especificação. Existem diversas formas de verificarmos se um software está correto. Elas são o **teste de programa**, a **inspeção** e a **prova de programas**.

O **teste de programa** permite identificarmos se um programa tem erros a partir da execução do programa de acordo com técnicas específicas.

A **inspeção** pode ser feita pela análise do código fonte e a sua execução mental com o auxílio de lápis e papel (**rastreamento**). Outra forma de inspeção é utilizando o *debugger* que permite acompanhar passo a passo a execução de cada instrução do programa e verificar o estado das variáveis de dados.

A **prova de programa** utiliza técnicas de lógica matemática que permite provar se um programa está correto em relação à especificação formal deste programa. Em um software complexo, se a sua arquitetura for especificada formalmente pode-se provar que cada componente individualmente está correto e que a sua integração também está correta.

A avaliação da **validação** do software visa determinar se a especificação do software (funcionalidade, arquitetura, interface, etc) satisfaz aos requisitos do usuário. Ela deve ser realizada durante toda a fase de desenvolvimento na medida que as especificações forem elaboradas.

A **avaliação da usabilidade** visa identificar as qualidades relacionadas com a interação entre o usuário e o software. Dentre estas qualidades estão a facilidade de aprendizado, facilidade de uso, produtividade, etc. A usabilidade é verificada de forma empírica através de **testes de usabilidade** que analisam o comportamento do usuário durante a interação.

Outras propriedades do software também precisam ser verificadas tais como **desempenho** e **robustez** e devem ser estudadas na literatura especializada.

2.2.3 As atividades da fase de operação

Durante a fase de operação do ciclo de vida o software deve ser instalado, utilizado e feita a sua manutenção.

A **instalação e configuração** é a atividade visa implantar o software no computador para que ele possa ser utilizado pelos usuários. A instalação requer que o software seja armazenado no lugar correto e que os *drivers* de dispositivos de hardware sejam configurados adequadamente para que o software funcione corretamente. Esta atividade pode ser realizada tanto por usuários como pelos engenheiros ou programadores do software.

A **utilização** é a atividade fim do software, uma vez que ele foi construído para auxiliar pessoas na realização de suas tarefas. A utilização deve seguir o modelo de interação especificado durante o seu design que deve está descrito no **manual de usuário**.

A **manutenção** é a atividade destinada a assegurar a qualidade do software durante a fase de operação. A manutenção se faz necessária para que o software possa ser transportado para outras plataformas de hardware e de sistemas operacionais, para satisfazer a novos requisitos dos usuários ou para corrigir eventuais erros. Ela pode envolver a **correção** de erros não detectados durante os testes, o **re-design** da funcionalidade, da interface de usuário, da arquitetura do software ou dos algoritmos de forma que novos requisitos sejam satisfeitos, e várias outras atividades.

2.3 Modelos do processo de desenvolvimento

Um modelo para um processo de desenvolvimento é uma proposta teórica que junto com o planejamento deve determinar quais atividades devem ser realizadas, quando, como e por quem. Nesta seção vamos apresentar alguns dos mais conhecidos modelos do processo de desenvolvimento. Eles são o modelo **Cascata**, o modelo **Evolutivo** ou Incremental, o modelo **Espiral** e o modelo **Transformação**.

A escolha de um destes modelos envolve diversos fatores. Um deles é o tipo de software - se é um software de banco de dados, um software de tempo-real, um software embutido, um sistema especialista. Outro fator importante é se a equipe de desenvolvimento é uma empresa de desenvolvimento (software house) ou se é a equipe de engenheiros da própria organização que utilizará o produto. A maneira como o produto será vendido e instalado é um outro fator importante - se o software é para ser vendido para um público amplo ou se será instalado em uma única empresa, sob encomenda.

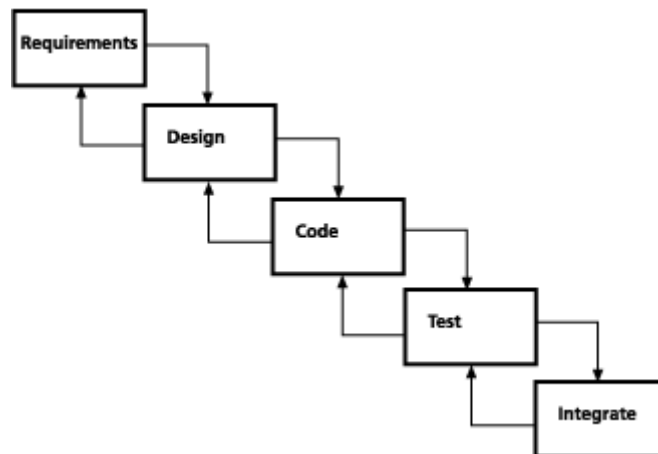
Para cada uma das atividades **métodos** específicos devem ser elaborados. Um conjunto de métodos relacionados entre si num esquema comum de acordo com um modelo é chamado de **metodologia** de desenvolvimento.

O Modelo Cascata

O modelo cascata tornou-se conhecido na década de 70 e é referenciado na maioria dos livros de engenharia de software ou manuais de padrões de software. Nele as atividades do processo de desenvolvimento são estruturadas numa cascata onde a saída de uma é a entrada para a próxima. As suas principais atividades são:

- estudo de viabilidade
- análise e especificação de requisitos
- design e especificação
- codificação e testes de componentes
- teste do sistema e integração
- entrega e instalação
- manutenção

Existem muitas variantes deste modelo propostas por diferentes pesquisadores ou empresas de desenvolvimento e adaptadas a diferentes tipos de software. A característica comum é um fluxo linear e seqüencial de atividades semelhantes a descritas anteriormente. A figura a seguir, mostra uma variante do modelo em cascata.



Este modelo, quando proposto, introduziu importantes qualidades ao desenvolvimento. A primeira chama a atenção de que o processo de desenvolvimento deve ser conduzido de forma disciplinada, com atividades claramente definidas, determinada a partir de um planejamento e sujeitas a gerenciamento durante a realização. Outra qualidade define de maneira clara quais são estas atividades e quais os requisitos para desempenhá-las. Por fim, o modelo introduz a separação das atividades da definição e design da atividade de programação que era o centro das atenções no desenvolvimento de software.

O modelo Cascata também é criticado por ser linear, rígido e monolítico. Inspirados em modelos de outras atividades de engenharia, este modelo argumenta que cada atividade apenas deve ser iniciada quando a outra estiver terminada e verificada. Ele é considerado monolítico por não introduzir a participação de clientes e usuário durante as atividades do desenvolvimento, mas apenas o software ter sido implementado e entregue. Não existe como o cliente verificar antecipadamente qual o produto final para detectar eventuais problemas.

Características particulares do software (ser conceitual, por exemplo) e a falta de modelos teóricos, técnicas e ferramentas adequadas mostram que é necessário haver maior flexibilidade neste fluxo seqüencial, permitindo volta atrás para eventuais modificações. Veremos mais adiante modelos que propõem maior flexibilidade no fluxo de execução.

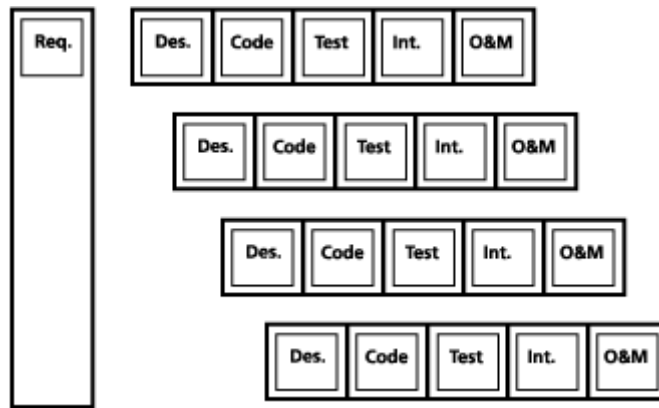
As métricas utilizadas nas estimativas de prazos e recursos humanos são ainda bastante imprecisas e quase sempre o planejamento de atividades precisa ser revisto. Normalmente os prazos não são cumpridos pois o planejamento, neste modelo, é feito nas etapas iniciais do desenvolvimento. A estrutura seqüencial e rígida também não permite que o planejamento seja refeito para corrigir falhas nas atividades de desenvolvimento.

O Modelo Evolutivo

O modelo evolutivo descreve um processo na qual o software deve ser desenvolvido de forma a evoluir a partir de protótipos iniciais. Para entender melhor este modelo é importante entender o que é [prototipação](#). A prototipação também aparece em outros modelo de processo.

O processo de desenvolvimento pode evoluir de diversas maneiras. Uma destas formas consiste em ir desenvolvendo partes funcionais de um software, isto é, que funcionam, mas não atendem a todos os requisitos por completo, e ir incrementando com novos pedaços até que todos os requisitos sejam atendidos. Para cada novo componente incorporado, as qualidades do protótipo devem ser verificadas experimentalmente.

Outra forma de evolução é iniciar o desenvolvimento com um protótipo rudimentar não funcional e torná-lo funcional ao longo do processo através do refinamento de seus componentes.



O fluxo de atividades do modelo evolutivo caracteriza-se por ser cíclico ou iterativo. Ele começa com o design e desenvolvimento de um protótipo inicial, que deve ser mostrado aos usuários e avaliado. Durante a avaliação novos requisitos são definidos e alterações e incrementos ao protótipo inicial devem ser feitas. Este ciclo deve ser repetido em direção ao produto final.

A grande vantagem deste modelo está em permitir a verificação antecipada do produto final por engenheiros, clientes e usuários, permitindo a correção dos problemas detectados.

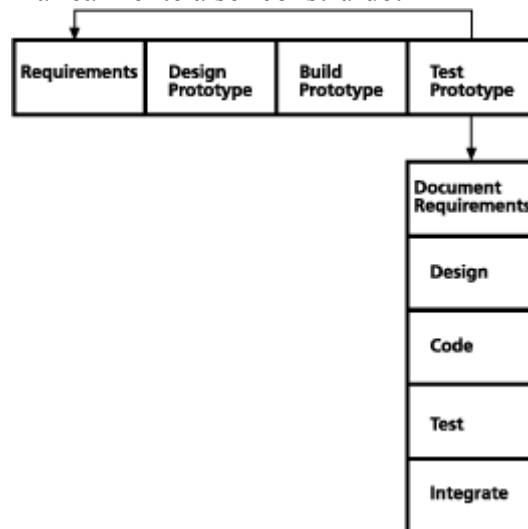
A extrema flexibilidade deste modelo e a falta de rigor leva a software que embora satisfaça aos requisitos dos usuários têm deficiências de desempenho, portabilidade, manutenção e outras qualidades internas.

Embora a prototipação tenha enormes vantagens e deva ser incentivada, basear o desenvolvimento no incremento de protótipos pode levar a software mal documentados e com arquiteturas mal definidas. Como os requisitos estão sempre sendo revistos a cada ciclo de desenvolvimento, torna-se praticamente impossível estimar custos e prazos e planejar as atividades de desenvolvimento.

A prototipação é mais adequada como método auxiliar à análise de requisitos e ao design do software, pois permite a validação antecipada do produto. Entretanto, não se deve deixar de lado o design da arquitetura de software e dos algoritmos para que ele possa ser construído utilizando uma linguagem de programação mais adequada.

Prototipação

Prototipação é uma abordagem baseada numa visão evolutiva do desenvolvimento de software, afetando o processo como um todo. Esta abordagem envolve a produção de versões iniciais - "protótipos" - de um sistema futuro com o qual pode-se realizar verificações e experimentações para se avaliar algumas de suas qualidades antes que o sistema venha realmente a ser construído.



Tipos de Protótipos

O relacionamento entre um protótipo e as atividades do processo de desenvolvimento - início do projeto e análise de requisitos, design da interface e da aplicação, e implementação - permite a identificação de quatro tipos de protótipos:

- **Protótipo de Apresentação** - oferece suporte ao início do projeto e é usado para convencer o cliente de que o futuro sistema é viável e que a interface do usuário se adequa aos requisitos. Na maioria dos casos é usado para mostrar visão que o usuário têm do sistema e revelar aspectos importantes da interface.
- **Protótipo Autêntico** - é um sistema de software provisório e funcional, geralmente projetado para ilustrar aspectos específicos da interface de usuários ou parte da funcionalidade, ajudando na compreensão dos problemas envolvidos.
- **Protótipo Funcional** -- é derivado do modelo do domínio do problema ou da especificação do software e serve para ajudar à equipe de desenvolvimento compreender questões relacionadas com a construção do sistema. Esse protótipo não interessa aos usuários.
- **Sistema Piloto** - é usado não apenas com propósitos ilustrativos, mas como um núcleo básico operacional do sistema. Esse sistema deve ser instalado no ambiente de aplicação e experimentado com os usuários.

Objetivos da Prototipação

Num projeto de software várias questões podem ser respondida com a construção de protótipos. Nas situações típicas de desenvolvimento podemos distinguir entre diferentes objetivos na prototipação:

- **Exploratória** - é quando o protótipo é usado para ajudar a esclarecer requisitos dos usuários com respeito ao sistema futuro. Uma prototipação também é exploratória quando se busca examinar uma variedade de opções de design de maneira a evitar a escolha de uma abordagem específica não adequada. Com esses objetivos os desenvolvedores adquirem informações sobre o domínio, os usuários e tarefas. Os usuários podem emitir informações e sugestões mais precisas, tornando-se parceiro das decisões que envolvem o desenvolvimento.
- **Experimental** - é quando a prototipação foca aspectos técnicos do desenvolvimento, oferecendo aos desenvolvedores resultados experimentais para tomada de decisões de design e implementação. Um aspecto essencial é a viabilização de uma base de comunicação entre os usuários e desenvolvedores para soluções de problemas técnicos de viabilidade e usabilidade, dentre outros. As principais vantagens para os desenvolvedores são a verificação e validação das decisões tomadas e soluções apresentadas.
- **Evolutiva** - A prototipação pode ser aplicada de maneira bastante proveitosa num processo de reengenharia em organizações, para avaliar o impacto que a introdução de novas tecnologias pode trazer. Nesse caso o protótipo não é visto apenas como uma ferramenta em projetos individuais, mas como parte de um processo contínuo de evolução dos processos organizacionais. Os desenvolvedores não são mais os protagonistas da prototipação, mas consultores que trabalham em cooperação com os usuários no processo de reengenharia.

Técnicas de prototipação

Do ponto de vista técnico, a prototipação pode ser conduzida segundo duas abordagens, relacionadas com técnicas específicas de construção. Experiências em desenvolvimento de software têm apontado inúmeras qualidades no design e implementação em diversas camadas. Essas camadas podem ir desde a interface de usuário à camadas mais básicas como uma base de dados e o sistema operacional. Dessa forma, podemos identificar duas formas de subdividir a prototipação.

- **Prototipação horizontal** - Apenas uma camada específica do sistema é construída. Por exemplo, a interface do usuário com suas janelas e *widgets*, ou camadas da aplicação como as funções para transação em bancos de dados.
- **Prototipação vertical** - Uma parte da funcionalidade do sistema é escolhida para ser implementada completamente. Esta técnica é apropriada quando aspectos da funcionalidade ainda estão em aberto.

Um outro critério para se classificar protótipo é de acordo com o relacionamento entre o protótipo e o sistema final. Em certos casos, o protótipo serve apenas para propósitos de especificação do sistema final e não será usado como parte integrante do sistema final. Ele é jogado fora. Um outro caso, é quando o protótipo vai sendo incrementado e se torna parte integrante (*building block*) do sistema. Por fim, o protótipo pode ser construído apenas para a compreensão de certos problemas que envolvem determinados interesses. Não existe a intenção de transformá-lo num sistema.

O Modelo Transformação

Um programa é uma descrição formal, isto é, ele é descrito por uma linguagem de programação cuja sintaxe e semântica são definidos formalmente. Apenas desta forma é que temos a garantia de que o programa será sempre executado da mesma forma pelo computador.

O modelo Transformação tem suas raízes na abordagem de métodos formais para o desenvolvimento de software. A idéia é que o desenvolvimento deve ser visto como uma seqüência de passos que gradualmente transforma uma especificação formal num programa. O passo inicial é transformar os requisitos informais numa especificação funcional formal. Esta descrição formal é então transformada numa outra descrição formal mais detalhada, e assim, sucessivamente, até chegar no ponto em que se tenha componentes de programas que satisfaçam a especificação. Durante este processo de transformações sucessivas - chamado de *otimização* - as especificações formais produzidas podem ser executadas por um processador abstrato, permitindo uma verificação formal da sua validação antes mesmo que o programa venha a ser implementado. Antes de serem transformadas cada especificação formal deve ser verificada em relação às expectativas dos clientes e usuários.

As transformações devem ser realizadas pelo engenheiro de software. A natureza formal da transformação possibilitam a aplicação de verificações matemáticas como prova, consistência e completude da especificação. As transformações podem ser realizadas de maneira automática por ferramentas que traduzem especificações formais mais abstratas em especificações mais detalhadas.

Este modelo prevê que o engenheiro de software deve ter a sua disposição uma biblioteca de componentes reutilizáveis que satisfaça especificações formais. Na *otimização*, à medida que as especificações de mais baixo nível vão sendo obtidas verifica-se quais componentes da biblioteca implementam parte desta especificação. Um ambiente automatizado de desenvolvimento (uma ferramenta CASE - Computer Aided Software Engineering) pode auxiliar este processo armazenando o histórico de decisões dos engenheiros de software.

O Modelo Espiral

O objetivo do modelo espiral é prover um *metamodelo* que pode acomodar diversos processos específicos. Isto significa que podemos encaixar nele as principais características dos modelos vistos anteriormente, adaptando-os a necessidades específicas de desenvolvedores ou às particularidades do software a ser desenvolvido. Este modelo prevê prototipação, desenvolvimento evolutivo e cíclico, e as principais atividades do modelo cascata.

Sua principal inovação é guiar o processo de desenvolvimento gerado a partir deste metamodelo com base em **análise de riscos** e um **planejamento** que é realizado durante toda a evolução do desenvolvimento. *Riscos* são circunstâncias adversas que podem surgir durante o desenvolvimento de software impedindo o processo ou diminuindo a qualidade do produto. São exemplos de riscos: pessoas que abandonam a equipe de desenvolvimento, ferramentas que não podem ser utilizadas, falha em equipamentos usados no desenvolvimento ou que serão utilizados no produto final, etc. A identificação e o gerenciamento de riscos é hoje uma atividade importantíssima no desenvolvimento de software devido à imaturidade da área e à falta de conhecimento, técnicas e ferramentas adequadas.

O modelo espiral descreve um fluxo de atividades cíclico e evolutivo constituído de quatro estágios.

- No estágio 1 devem ser determinados objetivos, soluções alternativas e restrições.
- No estágio 2, devem ser analisados os riscos das decisões do estágio anterior. Durante este estágio podem ser construídos protótipos ou realizar-se simulações do software.

- O estágio 3 consiste nas atividades da fase de desenvolvimento, incluindo design, especificação, codificação e verificação. A principal característica é que a cada especificação que vai surgindo a cada ciclo - especificação de requisitos, do produto, da arquitetura, da interface de usuário e dos algoritmos e dados - deve ser feita a verificação apropriadamente.
- O estágio 4 compreende a revisão das etapas anteriores o planejamento da próxima fase. Neste planejamento, dependendo dos resultados obtidos nos estágios anteriores - decisões, análise de riscos e verificação, pode-se optar por seguir o desenvolvimento num modelo Cascata (linear), Evolutivo ou Transformação. Por exemplo, se já no primeiro ciclo, os requisitos forem completamente especificados e validados pode-se optar por seguir o modelo Cascata. Caso contrário, pode-se optar pela construção de novos protótipos, incrementando-o, avaliando novos riscos e re-planejando o processo.

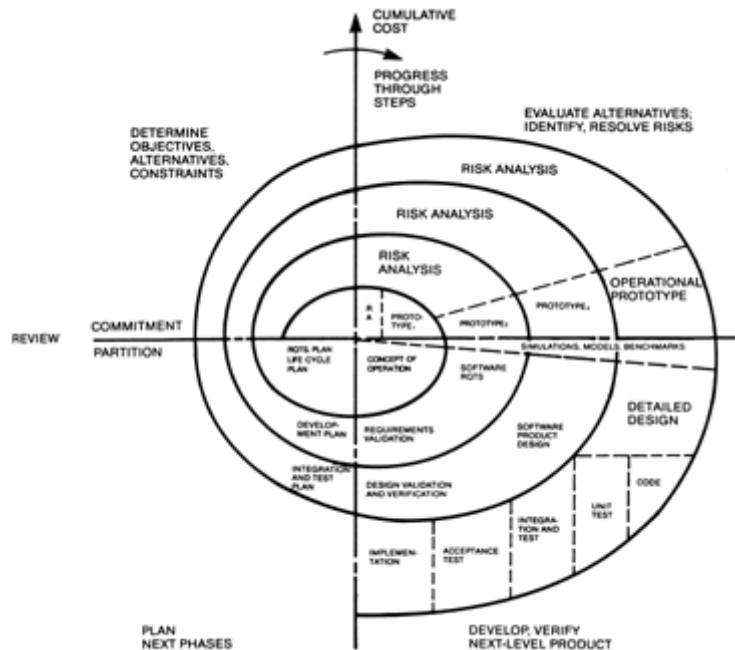


Figure 5. The Spiral model.

[Anterior](#) | [Índice](#) | [Próximo](#)

(C) Jair C Leite, 2000

Última atualização: 08/03/00